ProtoGen: Federated Privacy-Preserving Prototype Learning via Feature Generation in Heterogeneous Environments

Sujit Chowdhury, Aritra Bhaduri, Raju Halder

***Abstract*—Federated Learning (FL) enables collaborative model training across decentralized clients without sharing raw data. However, its performance declines when faced with heterogeneous data distributions across participating clients. To address this, existing FL frameworks often share not only locally trained model parameters but also additional client-specific information, which increases communication overhead and heightens privacy risks. In this paper, we propose ProtoGen, a novel FL framework that leverages privacy-preserving feature generation and prototype-based representation alignment to address these challenges in a unified and communication-efficient manner. Specifically, the framework introduces a Privacy-Preserving Generative Module (*PPGM*) that enables each client to train a lightweight generative model on differentially private feature embeddings and share its trained parameters with the server, eliminating the need to expose raw data or intermediate representations, thereby reducing communication cost and enhancing data privacy. To construct global class-wise prototypes, ProtoGen incorporates a novel server-side prototype construction mechanism that aggregates synthesized class-wise features using these shared generative model parameters for all clients. This design allows the clients to perform local feature alignment through the constructed global prototypes and to fine-tune the classifier layer via synthesizing underrepresented class features utilizing *PPGM*, enabling effective local model training across heterogeneous clients and substantially mitigating data distribution skew. Our experiments on benchmark datasets—MNIST, CIFAR-10, and CIFAR-100—show that ProtoGen consistently improves model convergence and test accuracy by an average of 4.76% over state-of-the-art FL methods, while significantly reducing communication overhead and preserving privacy.**

***Index Terms*—Federated Learning, Prototype Learning, Generative Model, Privacy Preserving, Data Heterogeneity**

I. Introduction

Federated Learning (FL) has gained prominence as an effective approach for collaboratively training machine learning models across decentralized datasets, ensuring data privacy by eliminating the need to share raw data between clients [1]. Despite its potential, FL performance tends to degrade significantly when data distributions across clients are highly heterogeneous or non-independent and identically distributed (non-IID) [2]. The heterogeneity in FL appears in various forms, such as label skewness, feature skewness, quantity skewness, which pose significant challenges for model training and generalization. [To combat with the effect of heterogeneity, recent research has explored various strategies, including variance reduction [3], [4], adaptive aggregation [5]–[10], feature distillation [11], and representation learning [12]–[14]. Notably, most of the proposed solutions under these strategies involve sharing additional information between clients and server such as control variates, client features, and classifier characteristics, which leads to increased communication costs and heightened risks of privacy breaches [15]–[17]. Whereas, prototype-based solution [18] has emerged as a promising alternative, since it shares only class-wise feature prototypes instead of full model parameters, thereby reducing communication overhead and exposure of sensitive information. However, as like others, the approach considers only a subset of clients - assuming that in practice few clients may not be available for training due to various issues like connectivity, power constraints, or computational limitations. As a result, the aggregated prototypes are biased toward the data distribution of active clients, neglecting information from inactive ones. This problem is further exacerbated under heterogeneous and imbalanced data distributions, where the absence of minority-class clients disproportionately skews the global representation and degrades generalization in subsequent rounds.]

[These observations point to a persistent research gap. FL systems need mechanisms that (i) construct global representations that remain *unbiased and temporally consistent* under partial participation, (ii) preserve privacy beyond raw data protection by preventing leakage of hidden features or gradients, and (iii) achieve these goals in a communication-efficient manner suitable for resource-constrained environments. Existing solutions typically address one of these aspects in isolation but compromise others. For instance, variance-reduction and adaptive aggregation approaches may improve stability but at high communication cost. Distillation-based methods enhance alignment but heighten privacy risks. Prototype learning improves efficiency but collapses under partial client participation. Thus, there is a clear need for a holistic framework that integrates unbiased prototype construction with privacy guarantees and communication efficiency. Motivated by these gaps, we focus on the following research questions (RQs):]

RQ1: How can a global representation in FL remain unbiased and temporally consistent under partial client participation?

RQ2: How can local training be decoupled from server-side aggregation in a way that preserves privacy yet avoids the trade-off of degraded training performance?

[Addressing these questions requires rethinking both client-side and server-side operations. Locally, models must align

Sujit Chowdhury, Aritra Bhaduri, Raju Halder are with the Department of Computer Science and Engineering, Indian Institute of Technology Patna, India, 801106.
E-mail: {sujit_2021cs35, aritra_2101ai40, halder}@iitp.ac.in

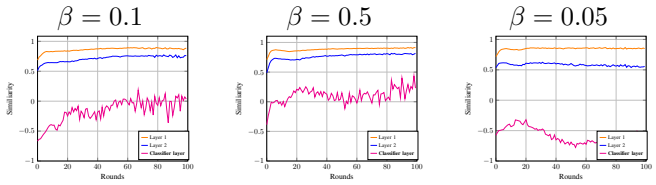


Fig. 1. Layer-wise similarity plots for ResNet-8 [19] across training rounds between FL clients under Dirichlet data distributions with $\beta = 0.1$, 0.5, and 0.05. Notably, the classifier layer shows high dissimilarity among clients, indicating significant classifier drift in extreme heterogeneous settings.

with global representations without leaking private information; globally, prototypes must integrate knowledge from both active and inactive clients to avoid bias.]

[In response to the above research questions, we propose ProtoGen (Federated Privacy-Preserving **Proto**type Learning via Feature **Gen**eration in Heterogeneous Environments), a novel FL framework that systematically eliminates prototype bias under partial participation while simultaneously enhancing privacy and reducing communication cost. At its core ProtoGen introduces a lightweight *class-wise feature generative module* at each client. This module learns to synthesize class-specific features that reflect the client's local distribution. On the client side, the generated features are used to augment missing classes and fine-tune classifier layers, thereby mitigating classifier drift and class imbalance in non-IID settings. On the server side, ProtoGen utilizes the shared generator parameters to synthesize class-wise features and construct global prototypes. Crucially, the server can regenerate features not only from currently active clients but also from inactive ones using their previously shared generators, yielding prototypes that are unbiased and temporally consistent across rounds, directly addressing *RQ1*. To ensure privacy without compromising model utility, ProtoGen *decouples local learning into two phases*: Phase-1 trains the main classifier on raw data to preserve predictive performance, while Phase-2 trains the generative module on noise-perturbed data, ensuring differential privacy. Only the generator parameters are shared, substantially reducing communication cost and protecting sensitive model information. This two-phase design resolves *RQ2*, as privacy is enforced without weakening the classification model. Together, these mechanisms allow ProtoGen to deliver robust global prototypes, privacy guarantees, and communication efficiency in one integrated framework.]

To summarize, the primary contributions of this paper are as follows:

1) We propose ProtoGen, a novel privacy-preserving FL framework that removes bias in global representation under partial participation in heterogeneous settings. By leveraging privacy-preserving feature generators instead of raw features or gradients coupled with a novel aggregation technique, ProtoGen ensures unbiased global representations while reducing communication and improves privacy guarantee while avoiding the performance trade-off.
2) We introduce a privacy-preserving feature generation module (*PPGM*) that enables both clients and the server to synthesize class-wise features under differential privacy (DP) guarantees. These synthetic features are used to fine-tune the clients' classifier layers and to construct global class-wise feature prototypes on the server.
3) We propose a novel server-side prototype construction mechanism that systematically aggregates differentially private, synthesized class-wise features to construct robust global class-wise prototypes. This approach effectively mitigates client drift and addresses the challenges of data heterogeneity in FL.
4) We present a detailed theoretical analysis of the proposed method, establishing its privacy guarantees and convergence behavior.
5) We demonstrate ProtoGen's superior performance compared to state-of-the-art methods through extensive experiments on benchmark datasets. The results demonstrate that ProtoGen achieves an average of 4.76% over state-of-the-art FL methods.

The rest of this paper is organized as follows: Section II discusses related work in the literature. Section III introduces the necessary preliminary background required to frame our approach. Section IV provides a detailed description of our proposed approach. Section V presents theoretical analysis of ProtoGen, proving its convergence and robustness. Section VI demonstrates performance evaluation of ProtoGen, along with a comparative summary w.r.t. state of the art approaches. Finally, Section VII concludes our work.

II. Related Work

Federated learning was first introduced in FedAvg [1] algorithm. This method allows clients to collaboratively train models without directly sharing their data, thus preserving privacy. However, despite the inherent privacy benefits, it presents several key challenges, particularly high communication costs and instability in model training due to data heterogeneity across clients. FedAvgM [5] integrates momentum into the global model updates, speeding up convergence in non-IID settings. While this reduces the number of communication rounds required, it introduces the need for careful hyperparameter tuning to maintain stability in diverse data environments. FedProx [20] introduced a proximal term in the local objective function to reduce client drift, which indirectly reduces communication by requiring fewer updates to converge. However, FedProx still struggles in extreme non-IID distribution settings. SCAFFOLD [4] attempts to directly address client drift using control variates to stabilize local updates, achieving better convergence with fewer rounds of communication, albeit with an increase in per-round communication overhead due to additional information exchanged between clients and server. FedDyn [3] uses dynamic regularization techniques to converge the client and server models to optimal values, but becomes unstable on extreme heterogeneous data across clients. MOON [21] tried to address the problem of heterogeneity using contrastive learning, however it also suffers from the same issues as FedDyn. Elastic [10] improves convergence through aggregation by interpolating client models according to its parameter sensitivity. However

this requires sharing of additional client's parameters sensitivity with the server and struggle in extreme heterogeneous settings. FedFA [22] is mainly used to address the issue of feature skewness by utilizing feature anchors but struggles in others heterogeneous settings. FedProto [13] addresses data heterogeneity in federated learning by aligning global feature distributions across clients. While this approach effectively improves model performance, it shares class-specific features, potentially exposing sensitive class information and making the system vulnerable to privacy attacks, such as feature inversion [17]. FedBN [23], on the other hand, applies client-specific batch normalization layers to handle feature skew, but fails to generalize in settings with severe data imbalance, leading to increased communication rounds as the global model struggles to converge. FedPVR [24] worked upon scaffold to improve the learning convergence in cases where client drift is observed. They were able to achieve promising results however their approach is not secure to privacy based attacks due to sharing of model parameters. Recent works such as FedFed [11] and FLUTE [14] explore more advanced techniques to address data heterogeneity. FedFed enhances model accuracy by performing feature augmentation across clients but introduces significant communication and computational overhead due to the need for sharing additional feature information. Similarly, FLUTE employs feature learning and classifier calibration to address heterogeneity, but it suffers from the need for extensive hyperparameter tuning and full client participation, which leads to increased communication demands. Another recent approach, pFedFDA [25] uses a global feature representation model and adapts to local distribution by computing the mean and variance of the data, and must share all these parameters to and fro between the server and clients. This creates a communication overhead, especially when there are many participating clients. In summary, while most existing approaches focus primarily on addressing data heterogeneity, they often introduce increased communication costs and increased risks of privacy breaches.

III. Preliminary

A. Federated Learning

Federated learning, introduced in [1] is a model training process where multiple parties collaborate to train a model using their private datasets. This is particularly useful as the parties do not need to share their data, which can be confidential and so is widely used among untrusted parties. In general this kind of learning involves a local model training where each party trains a model with their private data, followed by sharing of parameters to a central server. The central server then performs some kind of aggregation on the shared data and then re-distributes the aggregated parameter to all clients. All the clients then use the aggregated parameters to load a global model which is then used again for local training. It is to be noted that the parameters shared may or may not be the local model parameters, and the aggregation may or may not be simple averaging. The entire process from training of local models to aggregation at central server constitute a round. The next round of federated learning begins when client receives aggregated parameters from the server. The number of rounds is a hyperparameter that can be chosen as per requirement. As research increased in the domain of federated learning, it became evident that the original FedAvg is neither optimal for heterogeneous data nor privacy preserving in true sense as attacks like model inversion became known. The core mathematical operation behind FedAvg can be expressed as:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_i^k$$

where w_i^k denotes the model parameters from client k at round t, n_k is the size of client k's local dataset, $n = \sum_{k=1}^{K} n_k$ is the total data size across all clients, and w_{t+1} represents the updated global model parameters after aggregation.

B. Generative Model

A generative model is typically a model trained to generate simulated data. In the context of this work, it would mean generating features for a label or a class. The generative model should have two components - encoder and decoder. During training, the encoder takes the features and encodes them into a latent feature representation (which is μ and σ^2 in the case of VAE). The latent feature is then used to generate synthetic feature data using the decoder. The loss between actual and reconstructed features is used to train the model. During inference, synthetic class-guided latent features are fed into the decoder to generate corresponding class-wise feature representations. We introduce an encoder function E and a decoder function D to generate features conditioned on class labels. Given an input feature vector x (from space $\mathcal{X}$) and its class label y, the encoder compresses x into a latent representation z.

$$z = E(x)$$

The decoder then reconstructs the feature using both z and the label y:

$$\hat{x} = D(z, y)$$

This conditional decoder allows the generative model to tailor the reconstructed feature to the specified class. To train this model, we minimize a reconstruction loss that measures how closely the reconstructed feature $\hat{x}$ matches the original x. The most common loss function used is the mean square error:

$$\mathcal{L}_{rec} = ||x - \hat{x}||^2$$

summing (or averaging) over all feature dimensions.

C. Prototype Construction via Features Distillation

Prototype-based learning is a widely used technique in representation learning, where a prototype serves as a compact and discriminative summary of a class. These prototypes can be constructed using various methods such as class-wise averaging of feature embeddings, clustering-based approaches like K-means, or learned representations via attention mechanisms or contrastive objectives. In general, prototypes aim to capture the central tendency of class-specific feature distributions and are instrumental in tasks such as classification, few-shot

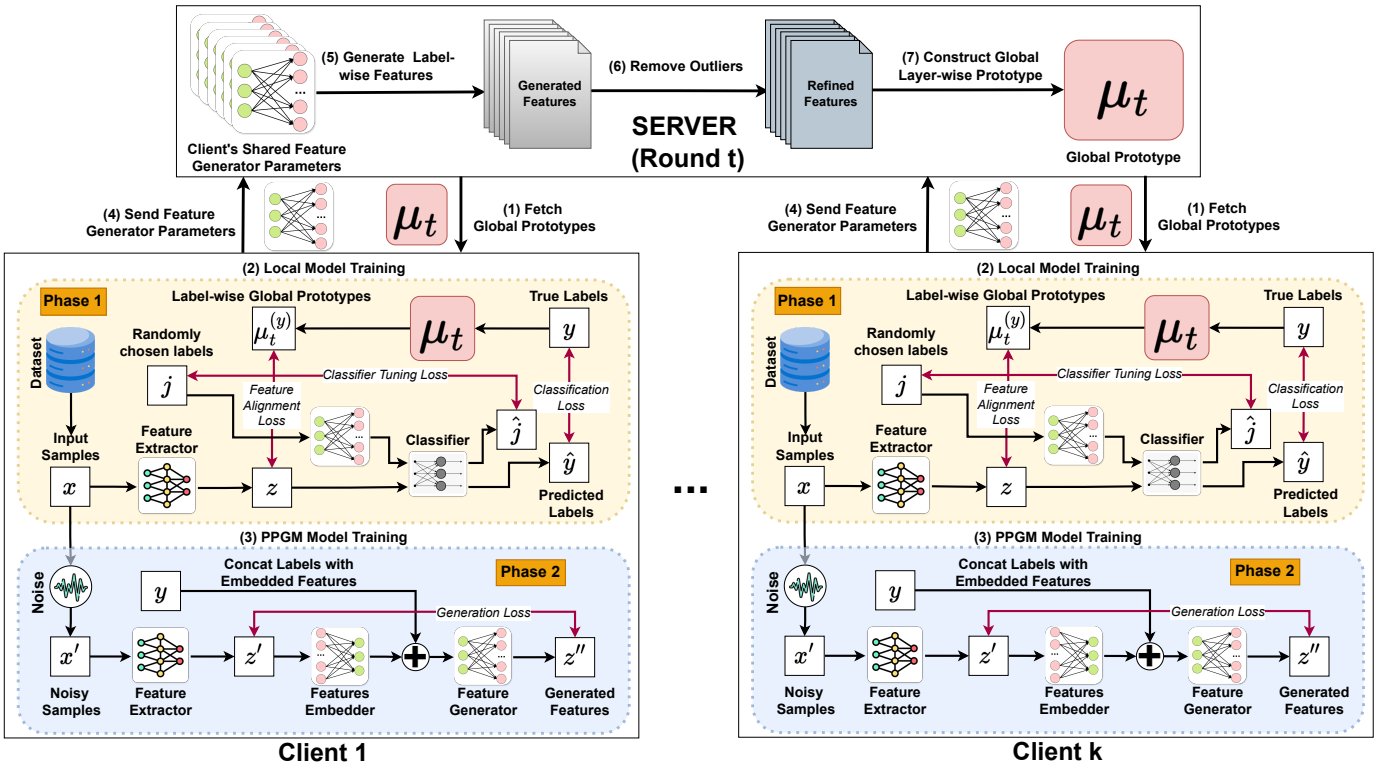


Fig. 2. An overview of ProtoGen learning framework. The learning process consists of the following steps: (1) clients fetch global prototypes from the server; (2) clients perform local model training with feature alignment and classification losses; (3) clients train the *PPGM* model using noisy local data; (4) clients send feature generator parameters to the server; (5) the server generates label-wise features using received generators; (6) outliers are removed from the generated features; and (7) the server constructs updated global prototypes for the next round.

learning, and knowledge distillation. A common formulation involves aggregating latent feature vectors corresponding to the same class. Given a set of feature embeddings $\mathbb{F}_c = \{\mathbf{f}_i^c\}_{i=1}^{N_c}$ associated with class c, the prototype $\mathbf{p}_c$ for class c is typically computed as the mean:

$$\mathbf{p}_c = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{f}_i^c \tag{1}$$

This aggregation-based approach provides a simple yet effective mechanism for capturing class-level semantics and serves as a foundation for various prototype-driven learning paradigms.

IV. ProtoGen: Proposed FL Approach

This section presents our proposed FL framework, ProtoGen, which adapts prototype learning to mitigate data heterogeneity by aligning local hidden features with the global class-wise features prototype. Figure 2 depicts the overall ProtoGen framework, which introduces the following key components: (1) a novel privacy-preserving feature generation module for each client to capture local hidden feature distribution from the local raw data without compromising privacy, and (2) a global class-wise feature prototype construction mechanism to obtain a generalized global representation via features distillation from client's shared feature generator parameters. In each round t, clients fetch μ_t from the server and train their local models in Phase 1 to minimize the local objective. Following this, in Phase 2, clients train the privacy-preserving generative module using hidden features ($\mathcal{Z}'$) extracted from noisy local data ($\mathcal{X}'$) and shares the associated features generator parameters ψ with the server. Finally, the server constructs global class-wise features prototype μ_{t+1} for the next round by distilling robust features from the shared ψ.

A. Problem Statement

Consider a federated learning environment with a set of clients $\mathbb{K}$, where each client $k \in \mathbb{K}$ has a local dataset $\mathbb{D}_k = \{(x_k^i, y_k^i)\}_{i=1}^{n_k}$, where n_k is the total number of data samples with client k. The data samples $x_k^i \in \mathcal{X}_k$ and labels $y_k^i \in \mathcal{Y}_k$ are drawn from client-specific distributions $p_k(x, y)$. In the non-IID setting, the distributions p_k can differ significantly across clients. The goal is to learn a generalized local model $\mathcal{F}(\theta_k, \mathcal{X}_k) \rightarrow \mathcal{Y}_k$ parameterized by θ_k that performs well under heterogeneous data distributions. This model consists of two main components: feature extractor and classifier. The feature extractor, parameterized by ϕ_k, transforms raw input data $\mathcal{X}_k$ into a feature space $\mathcal{Z}_k$, formally defined as: $\mathcal{H}(\phi_k, \mathcal{X}_k) \rightarrow \mathcal{Z}_k$. The classifier, parameterized by φ_k, takes the extracted feature representation $z = \mathcal{H}(\phi_k, x)$ to predict the corresponding class label, formally expressed as $\hat{y} = \mathcal{C}(\varphi_k, z)$, where $x \in \mathcal{X}_k, z \in \mathcal{Z}_k$. Additionally, clients train a privacy-preserving generative module $\mathcal{G}(\psi_k, \mathcal{Y}_k) \rightarrow \mathcal{Z}_k$, parameterized by ψ_k, is introduced to synthesize features that resemble those produced by $\mathcal{H}(\phi_k, x')$, where x' is the

noisy data. To enhance privacy, Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is added to the input, forming a perturbed version $x' = x + \epsilon$. The overall training objective is defined as follows:

$$\min_{\phi_k, \varphi_k, \psi_k} \mathbb{E}_{(x,y)\sim p_k(x,y)} [\mathcal{L}_{CE}(\mathcal{C}(\varphi_k, \mathcal{H}(\phi_k, x)), y) + \lambda_a \|\mathcal{H}(\phi_k, x) - \mu^{(y)}\|^2 + \mathcal{D}(\mathcal{G}(\psi_k, y), \mathcal{H}(\phi_k, x'))] \tag{2}$$

where $\mathcal{D}(\cdot, \cdot)$ is a similarity metric such as Mean Squared Error, $\mu^{(y)}$ is the global class-wise prototype for class label y received from the server, and λ_a controls the strength of feature alignment. $\mathcal{L}_{CE} : \hat{\mathcal{Y}} \times \mathcal{Y} \to \mathbb{R}_+$ is the local objective function, e.g. cross-entropy loss for classification task.

B. Client's Local Training

Each client $k \in \mathbb{K}$ performs local training in the following two phases:

Phase 1 (Minimizing Local Objective): In this phase, each client $k \in \mathbb{K}$ trains its local classification module through a combined objective function $\mathcal{L}_C$ that effectively balances three learning objectives: supervised classification, hidden features alignment, and synthetic feature-based classifier tuning. The supervised classification loss $\mathcal{L}_{CE}(\mathcal{C}(\varphi_{k,t}, \mathcal{H}(\phi_{k,t}, x)), y)$ is to ensure accurate predictions on local data, whereas hidden features alignment loss $\|\mathcal{H}(\phi_{k,t}, x) - \mu_t^{(y)}\|^2$ is to ensure that the input's hidden features $\mathcal{H}(\phi_{k,t}, x)$ align with the global class-wise feature prototypes μ_t received from the server. This alignment helps the client to mitigate the impact of data heterogeneity across clients via distilling global knowledge from class-wise feature prototypes. Additionally, to address label imbalance and mitigate classifier bias, each client randomly samples a class label $j \in \mathbb{C}$, where $\mathbb{C}$ denotes the global set of unique classes, and subsequently the client synthesizes representative features $\mathcal{G}(\psi_k^t, j)$, which are then passed to the classifier to compute an auxiliary classifier tuning loss $\mathcal{L}_{CE}(\mathcal{C}(\varphi_{k,t}, \mathcal{G}(\psi_{k,t}, j)), j)$. Utilizing the above three key learning objectives, we define the combined objective function $\mathcal{L}_C$ of local training as:

$$\mathcal{L}_C = \mathcal{L}_{CE}(\mathcal{C}(\varphi_{k,t}, \mathcal{H}(\phi_{k,t}, x)), y) + \lambda_a \|\mathcal{H}(\phi_{k,t}, x) - \mu_t^{(y)}\|^2 + \lambda_c \mathcal{L}_{CE}(\mathcal{C}(\varphi_{k,t}, \mathcal{G}(\psi_{k,t}, j)), j) \tag{3}$$

where $\mathcal{L}_{CE}$ represents the cross-entropy loss, $\mathcal{H}(\phi_{k,t}, x)$ is the feature extractor, $\mathcal{C}(\varphi_{k,t}, \cdot)$ is the classifier, and λa and λ_c are hyper-parameters controlling the importance of feature alignment and generated feature classification, respectively.

At each communication round t, the client updates its local model parameters $\theta_{k,t} = \{\phi_{k,t}, \varphi_{k,t}\}$, where ϕ_k^t and $\varphi_{k,t}$ represent feature extractor and classifier parameters, respectively. The parameters are optimized by minimizing the combined loss $\mathcal{L}_C$ via gradient descent with a local learning rate η_l, formally defined as:

$$\theta_{k,t+1} \leftarrow \theta_{k,t} - \eta_l \nabla_{\theta_{k,t}} \mathcal{L}_C \tag{4}$$

Phase 2: (Training of Privacy-Preserving Generative Module) To facilitate the sharing of client's knowledge while preserving privacy, each client trains a feature generator $\mathcal{G}(\psi, y)$.

Algorithm 1 Training Feature Generation Module

Input: Client dataset $\mathbb{D}_k$, feature generator $\mathcal{G}_k$, main model's feature extractor $\mathcal{H}_k$, hyper-parameters λ_{gen}, λ_{kl}, *PPGM* learning rate η_{gen}, and number of epochs E.
***PPGM* Model Training:**
for each local epoch $e = 1, 2, \ldots, E$ **do**
  for each batch $(x, y) \in \mathbb{D}_k$ **do**
    Add gaussian noise: $x' \leftarrow x + \mathcal{N}(0, \sigma^2)$
    Compute local feature: $z \leftarrow \mathcal{H}(\phi_k, x')$
    Compute generation Loss: $\mathcal{L}_{GEN} \leftarrow \lambda_{gen}\mathcal{D}(\mathcal{J}(\varrho_k, \mathcal{E}(\rho_k, z|y)|y), z) + \lambda_{kl}\text{KL}(\mathcal{E}_{\rho_k}(e|z, y)|p(e))$
    Update generative model parameters: $\psi_k^{e+1} \leftarrow \psi_k^e - \eta_{gen}\nabla_{\psi_k^e}\mathcal{L}_{GEN}$
  end for
end for
Return: Trained model parameters ψ_k

Gaussian noise ϵ is first added to input x to form x', and the feature extractor then maps x' to features $z' = \mathcal{H}(\phi, x')$. The generator is trained to approximate this mapping:

$$\min_{\psi} \mathbb{E}_{(x,y)\sim p_k(x,y)} [\mathcal{D}(\mathcal{G}(\psi, y), \mathcal{H}(\phi, x'))] \tag{5}$$

where $\mathcal{D}(\cdot, \cdot)$ is a similarity metric such as Mean Squared Error. Once training is complete, only the required feature generation parameters of $\mathcal{G}$ are shared with the server, preserving privacy (e.g., only decoder parameters are shared when using VAE as a generative module). This is worth noting that, even though this paper considers a Conditional Variational Autoencoder (CVAE) like architecture, the proposed framework is flexible enough to accommodate other generative models such as GANs, diffusion, etc., [26], [27].

CVAE for Feature Generation: To facilitate knowledge sharing while preserving privacy, each client trains a conditional variational autoencoder (CVAE) as the feature generator $\mathcal{G}(\psi, y)$. The CVAE learns to generate class-conditional features that closely resemble the distribution of real features extracted by the feature extractor $\mathcal{H}$. The CVAE consists of an encoder $\mathcal{E}(\rho, z|y)$ and a decoder $\mathcal{J}(\varrho, e|y)$, where ρ and ϱ are the encoder and decoder parameters, respectively. During training, Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is first added to input x to form x', and the feature extractor then maps x' to features $z' = \mathcal{H}(\phi, x')$. The encoder maps these features and class labels to a latent distribution parameterized by mean ν and variance σ^2, from which a latent code e is sampled using the reparameterization trick. The decoder then reconstructs the features conditioned on the latent code and class label. The training objective combines reconstruction loss and KL divergence:

$$\min_{\rho, \varrho} \mathbb{E}(x, y) \sim p_k(x, y) [\lambda_{gen}\mathcal{D}(\mathcal{J}(\varrho, \mathcal{E}(\rho, z|y)|y), z) + \lambda_{kl}\text{KL}(\mathcal{E}_\rho(e|z, y)|p(e))] \tag{6}$$

where $\mathcal{D}(\cdot, \cdot)$ is the mean squared error loss between the generated features and the features extracted from noise-perturbed inputs, $\text{KL}(\cdot|\cdot)$ represents the Kullback-Leibler divergence that encourages the learned latent distribution $\mathcal{E}_\rho(e|z, y)$ to be close

to a prior distribution $p(e)$ (typically a standard normal distribution), thereby promoting the generation of diverse features. The hyper-parameters λ_{gen} and λ_{kl} control the importance of each loss component.

Upon completion of generative module training, clients share only the decoder parameters ϱ with the server, preserving data privacy while enabling effective knowledge transfer through the learned feature distributions. This approach significantly reduces communication overhead compared to sharing full model parameters, as the decoder is typically much smaller than the complete model.

C. Server's Knowledge Distillation and Prototype Construction

At the server, the received feature generator parameters from all clients are used to synthesize class-wise feature representations. These synthetic features are then processed through an outlier removal mechanism to eliminate noisy or unrepresentative samples, resulting in a more reliable set of robust features. Using these robust features, the server constructs a global class-wise features prototype for each class $c \in \mathbb{C}$ by averaging features synthesized across contributing clients:

$$\mu = \left\{ \mu^{(c)} | \mu^{(c)} = \frac{1}{|\mathbb{K}|} \sum_{k \in \mathbb{K}} \frac{1}{|y^{(c)}|} \sum_{y_j \in y^{(c)}} \mathcal{J}(\varrho_k, y_j) \right\}_{c \in \mathbb{C}} \tag{7}$$

where $y^{(c)}$ denotes the list of labels c. The resulting global prototype set μ serves as a distilled representation of feature-level knowledge aggregated from the federation. It helps to mitigate the effects of data heterogeneity and enables consistent semantic alignment across clients.

In subsequent training rounds, the server distributes the global prototypes μ to the selected clients. Each client then uses these prototypes to guide the training of its local feature extractor. This is achieved by minimizing the discrepancy between its locally extracted features and the corresponding global class prototypes through the following alignment loss:

$$\mathcal{L}_{align} = \mathbb{E}_{(x,y)\sim p_k(x,y)} \left[\| \mathcal{H}(\phi, x) - \mu^{(y)} \|^2 \right] \tag{8}$$

This iterative process ensures that local models learn a unified feature space while maintaining privacy and minimizing communication overhead. As a result, ProtoGen effectively addresses data heterogeneity and enhances generalization in federated learning environments.

Algorithm 2 outlines the overall ProtoGen training process across multiple clients. The algorithm begins with initialization of local models and feature generators across all clients (line 1). For each communication round, the server randomly selects a subset of clients to participate for local training (line 4), and each selected client update their local model over multiple epochs (lines 6-13). During each epoch, clients sample batches from their local datasets, compute local features using the feature extractor, choose random class labels to mitigate classifier drift, generate synthetic features for the chosen class labels using the feature generator, and update local model parameters following Equation 4. After completing local model training, each client updates its feature generation module parameters following Algorithm 1 and sends only the generative portion of its updated parameters to the server (lines 14-15). The server then aggregates the received generators to construct updated global class-wise prototypes following Equation 7 (line 18). This process repeats until convergence or a maximum number of rounds is reached.

Algorithm 2 ProtoGen: Federated Privacy-Preserving Prototype Learning via Feature Generation

1: **Input:** Number of clients $\mathbb{K}$, number of communication rounds T, local datasets $\{\mathbb{D}_k | k \in \mathbb{K}\}$, local model learning rate η_l
2: **Federated Training:**
3: **for** each communication round $t = 1, \ldots, T$ **do**
4: **Select** a set of clients $\mathbb{S}_t$ for local training.
5: **Local Training:**
6: **for** each client $k \in \mathbb{S}_t$ **in parallel do**
7: **for** epocs $e = 1, \ldots, E$ **do**
8: **Sample** a batch of local dataset $\{x, y\} \in \mathbb{D}_k$
9: **Compute** local features: $z \leftarrow \mathcal{H}(\phi, x)$
10: **Choose** random labels: $j \in \mathbb{C}$
11: **Generate** synthetic features $z_{gen} \leftarrow \mathcal{G}(\psi_k, j)$
12: **Update** local model parameters following Equation 4.
13: **end for**
14: **Update** local generation module parameters $(\psi_{k,t})$ following Algorithm 1.
15: **Send** feature generator parameters $(\varrho_{k,t+1})$ to the server.
16: **end for**
17: **Aggregation:**
18: **Compute** global class-wise prototype following Equation 7.
19: **end for**

This design significantly improves communication efficiency by sharing only the feature generator parameters ϱ instead of the entire generative model parameters, which significantly reduces bandwidth requirements compared to traditional federated learning methods. Further, sharing of abstract generative capabilities rather than raw features from sensitive client data enhances privacy guarantees in the system. The bidirectional exchange of feature generators and class-wise prototypes fosters robust knowledge transfer across heterogeneous client distributions, effectively addressing key challenges in FL, such as non-IID data, class imbalance, and missing class scenarios. As a result, clients can leverage collective knowledge while retaining data privacy, promoting a more equitable and generalizable FL environment.

D. ProtoGen$^+$

To further reduce the communication cost of our approach, we propose an enhanced variant called ProtoGen$^+$. Instead of transmitting privacy-preserved feature generator parameters to the server, each client independently constructs class-wise feature prototypes using the locally trained *PPGM* module.

Specifically, client k constructs local class-wise prototypes μ_k as:

$$\mu_k = \left\{ \mu_k^{(c)} \,\middle|\, \mu_k^{(c)} = \frac{1}{n_k^{(c)}} \sum_{y_{k,j}^{(c)} \in \mathcal{D}_k^{(c)}} \mathcal{J}(\varrho_k, y_{k,j}^{(c)}) \right\}_{c \in \mathcal{C}} \tag{9}$$

where $\mathcal{D}_k^{(c)} = \{(x_{k,j}^{(c)}, y_{k,j}^{(c)}) \mid j = 1, 2, \ldots, n_k^{(c)}\}$ represents the subset of data samples belonging to class c at client k. Here, $x_{k,j}^{(c)}$ denotes the j-th feature vector of class c at client k, and $y_{k,j}^{(c)}$ its corresponding class label. $n_k^{(c)} = |\mathcal{D}_k^{(c)}|$ represents the number of samples of class i at client k. The function $\mathcal{J}(\varrho_k, y_{k,j}^{(c)})$ denotes the feature representation generated by the client's *PPGM* module's features generator parameters ϱ_k for label $y_{k,j}^{(c)}$. Once the local prototypes μ_k are constructed, they are transmitted to the server. The server then aggregates the received local prototypes across all participating clients $\mathcal{K}$ to compute global class-wise prototypes μ as follows:

$$\mu = \left\{ \mu^{(c)} \,\middle|\, \mu^{(c)} = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \mu_k^{(c)} \right\}_{c \in \mathcal{C}} \tag{10}$$

where, $\mu_k^{(c)}$ denotes the prototype for class c from client k. The resulting global class-wise features prototypes μ are then shared with clients for the subsequent round of federated learning.

This prototype-based communication scheme not only reduces the communication cost per round but also significantly lowers the server-side computation overhead. Unlike the original ProtoGen, where the server had to regenerate class-wise features from each client's generator parameters, ProtoGen$^+$ simplifies the process by directly using aggregated prototypes. However, this comes at the cost of some compromise in privacy as during local-prototype sharing clients will reveal which class samples are present in their corresponding local trainset.

V. Theoretical Analysis of ProtoGen

This section presents the following theorems, which collectively demonstrate the theoretical foundations of ProtoGen.

A. Preliminaries and Notation

Let $\mathbb{K}$ be the set of all clients, with $|\mathbb{K}| = K$. For each client $k \in \mathbb{K}$, let $\mathbb{D}_k$ be its local dataset, with $n_k = |\mathbb{D}_k|$ and $n = \sum_{k=1}^{K} n_k$. Let $\theta_{k,t} \in \mathbb{R}^d$ denote the local model parameters of client k at round t. Let $\rho_{k,t} \in \mathbb{R}^p$ and $\varrho_{k,t} \in \mathbb{R}^q$ denote the *PPGM* encoder and decoder parameters of client k at round t, respectively. $\psi_k \leftarrow \{\rho_k, \varrho_k\}$ represents the general *PPGM* model parameters of client k.

We define the following:

- $\mathcal{L}_C(\theta; x, y)$: loss function for client models
- $\mathcal{L}_{GEN}(\psi; x, y)$: loss function for *PPGM*
- $\mathcal{T}(\theta_k) = \mathbb{E}_{(x,y)\sim\mathbb{D}_k}[\mathcal{L}_C(\theta_k; x, y)]$: expected combined local objective loss for client k
- $\mathcal{V}(\psi_k) = \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}_{(x,y)\sim\mathbb{D}_k}[\mathcal{L}_{GEN}(\psi_k; x, y)]$: expected feature generation loss for client k.

B. Assumptions

We make the following assumptions:

Assumption 1 (Smoothness): $\mathcal{T}(\theta_k)$ is L-smooth for all k: $\forall \theta_k, \theta_k' \in \mathbb{R}^d$,

$$\|\nabla\mathcal{T}(\theta_k) - \nabla\mathcal{T}(\theta_k')\| \leq L_1 \|\theta_k - \theta_k'\| \tag{11}$$

where $L_1 > 0$ is the Lipschitz constant for the local model.

Assumption 2 (Strong Convexity): $\mathcal{T}(\theta_k)$ is μ-strongly convex for all k: $\forall \theta_k, \theta_k' \in \mathbb{R}^d$,

$$\mathcal{T}(\theta_k') \geq \mathcal{T}(\theta_k) + \langle \nabla\mathcal{T}(\theta_k), \theta_k' - \theta_k \rangle + \frac{\mu}{2}\|\theta_k' - \theta_k\|^2 \tag{12}$$

Assumption 3 (Bounded Variance): The variance of stochastic gradients is bounded for both client models and *PPGM*, $\forall k \in \mathbb{K}, \forall \theta_k \in \mathbb{R}^d$:

$$\mathbb{E}_{(x,y)\sim\mathbb{D}_k}[\|\nabla\mathcal{L}_C(\theta_k; x, y) - \nabla\mathcal{T}(\theta_k)\|^2] \leq \sigma_1^2 \tag{13}$$

$$\mathbb{E}_{(x,y)\sim\mathbb{D}_k}[\|\nabla\mathcal{L}_{GEN}(\psi_k; x, y) - \nabla\mathcal{V}(\psi_k)\|^2] \leq \sigma_2^2 \tag{14}$$

*Assumption 4 (*PPGM *Lipschitz Continuity):* The *PPGM* model is Lipschitz continuous with respect to its parameters:

$$\|\nabla\mathcal{V}(\psi_k) - \nabla\mathcal{V}(\psi_k')\| \leq L_2(\|\psi_k - \psi_k'\|) \tag{15}$$

where $L_2 > 0$ is the Lipschitz constant for the *PPGM* model.

Assumption 5 (Gradient Dissimilarity): For each client $k \in \mathbb{K}$, assume that the loss function $\mathcal{T}(\theta_k)$ is differentiable and bounded below, with $\theta_{k,t}$ denoting the model parameters at round t, and θ_k^* being a minimizer of $\mathcal{T}(\theta_k)$. Then, there exist constants $G \geq 0$ and $\beta \geq 0$ such that:

$$\frac{1}{K}\sum_{k=1}^{K} \|\nabla\mathcal{T}(\theta_{k,0}) - \nabla\mathcal{T}(\theta_k^*)\|^2 \leq G^2 + 2\beta\left(\mathcal{T}(\theta_{k,0}) - \mathcal{T}(\theta_k^*)\right) \tag{16}$$

where G^2 is a non-negative constant that upper bounds the inherent variance in the client gradients (even near local optima), β is a non-negative constant that quantifies how gradient differences grow with suboptimality (i.e., how far $\theta_{k,t}$ is from θ_k^* in terms of objective value).

C. Convergence Analysis

Lemma 1 (One-step Progress for Client Models): For client $k \in \mathbb{K}$, gradient computed with learning rate η_l over E local epochs at round $t \geq 0$, its one-step convergence bound follows:

$$\begin{aligned}\mathbb{E}[\|\theta_{k,t+1} - \theta_k^*\|^2] \leq (1 - \mu\eta_l)\mathbb{E}[\|\theta_{k,t} - \theta_k^*\|^2] \\ - 2\eta_l\mathbb{E}[\mathcal{T}(\theta_{k,t}) - \mathcal{T}(\theta_k^*)] \\ + \eta_l^2\left(\sigma_1^2 + 6L\gamma_1 E\eta_l + 8(E-1)^2\gamma_1^2\right)\end{aligned} \tag{17}$$

where $\gamma_1 = \max\{\frac{8L_1}{\mu}, E\}$

Proof 1: Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

Lemma 2 (One-step Progress for PPGM*):* For any round $t \geq 0$, the convergence bound for *PPGM* follows:

$$\begin{aligned}\mathbb{E}[\|\psi_{k,t+1} - \psi_k^*\|^2] \leq \mathbb{E}[\|\psi_{k,t} - \psi_k^*\|^2] \\ - 2\eta_g\mathbb{E}[\mathcal{V}(\psi_{k,t}) - \mathcal{V}(\psi_k^*)] \\ - \frac{\eta_g}{L_2}\sigma_2^2 + \eta_g^2\sigma_2^2\end{aligned} \tag{18}$$

Proof 2: Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

Theorem 1 (Convergence of ProtoGen in Convex Settings): Let Assumptions 1-4 hold. Let the learning rates be set as $\eta_l = \frac{2}{\mu(t+\gamma_1)}$ for client models and $\eta_g = \frac{2}{L_2(t+\gamma_2)}$ for the *PPGM*, where $\gamma_1 = \max\{\frac{8L_1}{\mu}, E\}$, $\gamma_2 = \max\{\frac{8L_2}{\mu}, E\}$, and E is the number of local epochs. Then, for $T \geq 1$ number of global rounds of training, the output of ProtoGen satisfies, following convergence bound:

$$\mathbb{E}[\mathcal{T}(\bar{\theta}_{k,T})] - \mathcal{T}(\theta_k^*) \leq \frac{4L_1\gamma_1}{\mu T}\left(1 + \log\left(\frac{T}{\gamma_1}+1\right)\right) \tag{19}$$

$$\mathbb{E}[\mathcal{V}(\bar{\psi}_{k,T})] - \mathcal{V}(\psi_k^*) \leq \frac{4L_2\gamma_2}{T}\left(1 + \log\left(\frac{T}{\gamma_2}+1\right)\right) \tag{20}$$

where $\bar{\theta}_{k,T} = \frac{1}{T}\sum_{t=1}^{T}\theta_{k,t}$, $\bar{\psi}_T = \frac{1}{T}\sum_{t=1}^{T}\psi_t$, and θ_k^*, ψ^* are the respective optimal parameters.

Proof 3 (Theorem 1 (Client Model Bound)): Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

Theorem 2 (Non-Convex Convergence Bound for Client Model): Let Assumptions 1–4 hold. Suppose a fixed client k performs local SGD with step size η_l for T rounds, each consisting of E local updates. Then, the iterates $\{\theta_{k,t}\}_{t=0}^{T-1}$ satisfy:

$$\sum_{t=0}^{T-1}\mathbb{E}[\|\nabla\mathcal{T}(\theta_{k,t})\|^2] \leq \frac{2}{\eta_l}\left(\mathcal{T}(\theta_{k,0}) - \mathcal{T}(\theta_k^*)\right) + L_1 T\eta_l\sigma_1^2. \tag{21}$$

Proof 4 (Theorem 2 (Non-Convex Bound)): Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

D. Privacy Analysis of ProtoGen

Let each client maintain a feature extractor $\mathcal{H}(\phi_k, x)$ and a *PPGM* model with encoder $\mathcal{E}(\rho_k, z|y)$ and decoder $\mathcal{J}(\varrho_k, e|y)$. The privacy mechanism relies on input perturbation, followed by feature extraction and training a generator only on the perturbed representations.

Assumption 6 (Input Perturbation by Gaussian Noise): Each input x_i is perturbed using additive Gaussian noise:

$$\tilde{x}_i = x_i + \xi_i, \quad \text{where } \xi_i \sim \mathcal{N}(0, \sigma^2 I_d).$$

Assumption 7 (Lipschitz Continuity of the Feature Extractor): The feature extractor $\mathcal{H}$ is L-Lipschitz continuous, i.e., for all $x, x' \in \mathbb{R}^d$,

$$\|\mathcal{H}(\phi_k, x) - \mathcal{H}(\phi_k, x')\|_2 \leq L \cdot \|x - x'\|_2.$$

Definition 1 (Differential Privacy [28]): A randomized algorithm $\mathcal{M}$ satisfies (ε, δ)-differential privacy if for any two adjacent datasets $\mathcal{D}, \mathcal{D}'$ differing in at most one record, and for any measurable subset S of outputs,

$$\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^{\varepsilon}\Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta.$$

Definition 2 (Global Sensitivity): The L_2-sensitivity of a function $\mathcal{H}(\phi_k, \mathcal{X}_k) \to \mathcal{Z}_k$ is

$$\Delta_h = \sup_{\mathcal{D},\mathcal{D}' \text{ differ in one sample}} \|\mathcal{H}(\phi_k, \mathcal{D}) - \mathcal{H}(\phi_k, \mathcal{D}')\|_2.$$

Theorem 3 (Gaussian Mechanism [28]): Let $\mathcal{H}(\phi_k, \mathcal{X}_k) \to \mathcal{Z}_k$ be a function with L_2 sensitivity Δ_h. Then the mechanism

$$\mathcal{M}(\mathcal{D}) = \mathcal{H}(\phi_k, \mathcal{D}) + \mathcal{N}(0, \sigma^2 I_d)$$

is (ε, δ)-differentially private for any $\delta > 0$, where

$$\varepsilon = \frac{\Delta_f^2}{2\sigma^2} + \log(1/\delta).$$

Lemma 3 (Differential Privacy of Noisy Feature Extraction): Let $\mathcal{M}(x) = \mathcal{H}(\phi_k, x + \xi)$ where $\xi \sim \mathcal{N}(0, \sigma^2 I_d)$, and let the input sensitivity $\Delta_x := \sup_{x,x'} \|x - x'\|_2$ be bounded. Then $\mathcal{M}$ satisfies (ε, δ)-differential privacy with

$$\varepsilon = \frac{L^2\Delta_x^2}{2\sigma^2} + \log(1/\delta).$$

Proof 5: Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

Lemma 4 (Post-Processing Invariance [28]): Let $\mathcal{M} : \mathcal{D} \to \mathcal{Y}$ be an (ε, δ)-DP mechanism. Let $f : \mathcal{Y} \to \mathcal{Z}$ be any (possibly randomized) function. Then $f \circ \mathcal{M}$ also satisfies (ε, δ)-DP.

Theorem 4 (Privacy Guarantee of PPGM *Training and Sharing):* Let each feature z_i used for *PPGM* model training be generated via $z_i = \mathcal{H}(\phi_k, x_i + \xi_i)$, where $\xi_i \sim \mathcal{N}(0, \sigma^2 I_d)$ and $\mathcal{H}$ is L-Lipschitz. Then:

1) The feature memory $\tilde{\mathcal{D}}_k = \{(\tilde{z}_i, y_i)\}_{i=1}^m$ satisfies (ε, δ)-differential privacy with respect to the original data $\mathbb{D}_k$.
2) The decoder $\mathcal{J}_{\varrho_k}$ trained on $\tilde{\mathcal{D}}_k$ also satisfies (ε, δ)-DP due to post-processing.

Proof 6 (Theorem 4): Detailed proof is reported in the supplementary document attached with this manuscript, due to space constraints.

VI. Experiment Evaluation

A. Implementation

We implement our proposed method and all baselines in Python 3.12 using PyTorch. For local classification, we employ three architectures: LeNet-5 [29] (a 7-layer CNN with 5×5 convolutions, tanh activations, and average pooling), ResNet-8 [19] (a lightweight residual network with three residual blocks), and ResNet-18 [19] (a deeper residual model with four residual stages and global average pooling). Additionally, each client trains a conditional variational autoencoder based privacy-preserving generative model (*PPGM*) consisting of an encoder with two fully connected layers (batch normalization + ReLU activations), taking as input concatenated feature embeddings and one-hot embedding of class labels, and a symmetric decoder for feature reconstruction. All models are optimized using SGD with standard settings, and experiments are conducted using NVIDIA A100 GPUs.

TABLE I

PERFORMANCE COMPARISON ON THREE BENCHMARK DATASETS UNDER VARYING DIRICHLET β VALUES AND LABEL NOISE LEVELS. ALL EXPERIMENTS ARE RUN FOR 200 GLOBAL ROUNDS. WE REPORT THE NUMBER OF ROUNDS TO REACH THE CONVERGENCE CRITERION (CR), THE HIGHEST AVERAGE TRAINING ACCURACY (TRAIN ACC), AND THE HIGHEST AVERAGE TEST ACCURACY (TEST ACC). THE BEST RESULTS FOR EACH DATASET AND SETTING ARE HIGHLIGHTED IN BOLD.

Dataset	Method	# of Comm Params ($\times 10^3$)	$\beta = 0.5$			$\beta = 0.1$			$\beta = 0.05$		
			CR	Train ACC	Test ACC	CR	Train ACC	Test ACC	CR	Train ACC	Test ACC
MNIST	FedAvg	993	200	99.29	97.65	200	99.19	97.25	200	98.89	97.18
	FedProx	993	200	99.29	97.68	200	99.19	97.25	200	98.95	97.34
	FLUTE	993	200	99.77	97.52	200	99.89	97.85	200	99.68	96.41
	FedProto	**18**	143	99.9	96.23	194	99.91	96.85	177	99.9	98.13
	SCAFFOLD	1986	146	99.9	97.83	200	99.64	97.42	200	99.43	98.08
	MOON	993	200	97.88	95.65	200	95.78	94.68	200	98.68	96.85
	FedPVR	1011	200	99.65	97.64	200	99.84	97.48	200	99.32	97.24
	ProtoGen	482	**114**	99.9	**98.03**	**150**	99.9	**98.15**	**154**	**99.92**	**98.48**
	ProtoGen$^+$	**18**	132	**99.92**	97.86	178	**99.92**	98.05	170	99.9	98.38
CIFAR-10	FedAvg	6453	200	79.6	65.38	200	72.09	53.8	200	66.25	49.45
	FedProx	6453	200	79.85	67.32	200	72.58	52.18	200	65.19	49.91
	FLUTE	6453	200	69.77	64.65	200	64.92	58.92	200	62.43	56.32
	FedProto	**27**	106	99.9	66.23	98	99.91	72.85	65	99.9	75.18
	SCAFFOLD	12906	200	89.59	62.79	200	89.44	68.42	200	82.97	69.18
	MOON	6453	200	73.16	66.59	200	65.81	56.51	200	61.09	54.06
	FedPVR	6480	200	80.52	59.78	200	79.84	67.48	200	78.32	68.23
	ProtoGen	605	**83**	**99.96**	**66.83**	**71**	**99.93**	**73.91**	**59**	99.9	**76.8**
	ProtoGen$^+$	**27**	98	99.92	66.46	89	99.9	73.68	67	**99.94**	75.97
CIFAR-100	FedAvg	235784	139	99.9	27.51	157	99.9	25.65	149	99.9	23.45
	FedProx	235784	98	99.9	26.68	200	99.48	26.31	200	99.62	26.22
	FLUTE	235784	200	98.36	25.74	200	91.22	24.73	200	90.91	23.48
	FedProto	**1076**	37	99.9	26.31	52	99.92	31.44	54	99.9	32.31
	SCAFFOLD	471568	66	99.9	25.45	101	99.9	22.38	116	99.91	24.18
	MOON	235784	98	99.91	25.65	200	99.84	30.18	200	92.68	25.94
	FedPVR	236860	93	99.9	25.03	150	99.95	25.46	190	99.96	23.23
	ProtoGen	2814	**29**	99.9	**27.83**	**46**	**99.98**	**34.69**	**46**	**99.98**	36.21
	ProtoGen$^+$	**1076**	35	**99.92**	27.24	**46**	99.9	34.43	**46**	99.93	**36.35**

B. Experiment Setup

To comprehensively assess the performance of our approach, we compare ProtoGen and ProtoGen$^+$ against seven state-of-the-art federated learning methods: FedAvg, FedProx, FLUTE, SCAFFOLD, FedProto, MOON, and FedPVR. These baselines span a diverse set of strategies addressing challenges in federated optimization, personalization, and representation learning. Our experiments are performed on three widely-used datasets: MNIST, CIFAR-10, and CIFAR-100, each presenting unique challenges and characteristics. To investigate the effectiveness of ProtoGen and the baseline methods under different model architectures, we employ a variety of neural networks for local training. Specifically, for the MNIST dataset, we utilize the LeNet-5 model, which is well-suited for handwritten digit recognition. For the more complex CIFAR-10 and CIFAR-100 datasets, we employ ResNet-8 and ResNet-18 models, respectively, due to their demonstrated effectiveness in image classification tasks. To ensure fairness and reproducibility of our comparisons, we maintain fixed random seeds and consistent settings across all experiments. This allows us to isolate the impact of federated learning algorithms on model performance, minimizing the influence of random variations. For ProtoGen, we tune the hyperparameters λ_a and λ_c over the range $[0, 1]$, and find that $\lambda_a = 0.2$ and $\lambda_c = 0.3$ yield the best performance. In training the *PPGM* model, we use the Adam optimizer with a learning rate of $\eta_{PPGM} = 0.001$. For all experiments, the local model training is configured with a learning rate $\eta_l = 0.01$, a batch size of 64, and 5 local training epochs. In FLUTE, out of 5 local epochs, the initial 3 epochs update both the feature representation and the local classification head, while in the remaining 2 epochs, only the local head is updated. These values were selected based on extensive hyperparameter tuning to ensure optimal performance. For all other baselines, we adopt the optimal hyperparameter configurations as recommended in their respective proposals. Each experiment runs for up to 200 communication rounds, with an early stopping criterion applied when the average training loss across all clients falls below 0.05. We empirically choose this threshold as we observe that once the training loss drops below 0.05, the corresponding training accuracy typically reaches approximately 99.9%, indicating convergence. The choice of 200 communication rounds is motivated by the need to analyze the convergence behavior of all baseline methods. Empirically, we find that most approaches converge well within this range, making it a suitable upper bound for fair and consistent evaluation across different methods and datasets. Additionally, the experiments were performed under a variety of data heterogeneity scenarios, specifically label skew (controlled by Dirichlet [30] parameter β), and fixed class-per-client (represented by $\#FC = x\%$, where x denotes the percentage of total class across all clients' datasets).

C. Performance Analysis

Tables I and II present a comprehensive performance evaluation of ProtoGen and ProtoGen$^+$ against popular state-of-the-art federated learning methods under two types of data

TABLE II
PERFORMANCE COMPARISON ON THREE BENCHMARK DATASETS UNDER FIXED CLASS-PER-CLIENT SETTINGS, REPRESENTED BY $\#FC = x\%$. ALL EXPERIMENTS ARE RUN FOR 200 GLOBAL ROUNDS. WE REPORT THE NUMBER OF ROUNDS TO REACH THE CONVERGENCE CRITERION (CR), THE HIGHEST AVERAGE TRAINING ACCURACY (TRAIN ACC), AND THE HIGHEST AVERAGE TEST ACCURACY (TEST ACC). THE BEST RESULTS FOR EACH DATASET AND SETTING ARE HIGHLIGHTED IN BOLD.

Dataset	Method	# of Comm Params ($\times 10^3$)	$\#FC = 20\%$			$\#FC = 40\%$			$\#FC = 60\%$		
			CR	Train ACC	Test ACC	CR	Train ACC	Test ACC	CR	Train ACC	Test ACC
MNIST	FedAvg	993	200	98.89	97.68	200	99.23	97.95	200	99.31	98.01
	FedProx	993	200	98.93	97.79	200	99.47	98.25	200	99.68	98.16
	FLUTE	993	200	99.77	98.15	200	99.19	97.85	200	98.98	97.69
	FedProto	**18**	145	99.9	98.83	119	99.9	98.74	91	99.9	98.61
	SCAFFOLD	1986	200	99.89	98.13	139	99.92	98.37	124	99.97	98.43
	MOON	993	200	97.88	96.79	200	98.35	97.29	200	98.68	97.85
	FedPVR	1011	200	99.47	98.04	198	99.9	98.18	189	99.91	98.38
	ProtoGen	482	**140**	99.9	98.98	**106**	99.92	**98.81**	**77**	**99.92**	**98.68**
	ProtoGen$^+$	**18**	142	**99.91**	**99.04**	113	**99.94**	98.79	86	99.9	98.63
CIFAR-10	FedAvg	6453	200	70.26	56.89	200	81.09	65.58	200	66.25	49.45
	FedProx	6453	200	70.85	56.62	200	79.98	65.18	200	65.19	49.91
	FLUTE	6453	200	86.77	66.65	200	84.92	65.92	200	78.43	67.32
	FedProto	**27**	125	99.94	79.23	116	99.9	70.3	65	99.9	68.82
	SCAFFOLD	12906	200	80.6	69.12	200	89.81	62.47	200	82.97	69.18
	MOON	6453	200	79.31	67.62	200	85.94	65.65	200	80.09	64.06
	FedPVR	6480	200	70.52	63.4	200	78.11	61.5	200	78.32	68.23
	ProtoGen	605	**94**	**99.97**	**81.07**	**82**	**99.95**	69.91	**59**	99.9	69.28
	ProtoGen$^+$	**27**	98	99.92	80.21	93	99.93	**70.59**	61	**99.94**	**69.86**
CIFAR-100	FedAvg	235784	113	99.9	25.38	93	99.94	25.96	89	99.91	27.8
	FedProx	235784	113	99.91	25.58	93	99.48	26.31	89	99.62	27.82
	FLUTE	235784	200	92.41	25.89	200	95.34	27.61	200	86.64	27.48
	FedProto	**1076**	53	**99.97**	27.86	45	99.92	27.72	41	99.9	27.78
	SCAFFOLD	471568	75	99.94	24.51	63	99.9	28.23	60	99.91	26.91
	MOON	235784	113	99.9	24.41	93	99.84	27.72	84	92.68	26.48
	FedPVR	236860	103	99.91	24.05	90	99.9	27.92	78	**99.96**	26.12
	ProtoGen	2814	**47**	**99.97**	**29.03**	43	**99.98**	**28.88**	**32**	99.92	**28.6**
	ProtoGen$^+$	**1076**	52	99.91	29.31	**42**	99.94	28.73	38	99.9	28.45

heterogeneity: (1) label distribution skew, simulated using a Dirichlet distribution with concentration parameters $\beta \in \{0.5, 0.1, 0.05\}$ corresponding to mild, moderate, and extreme heterogeneity, respectively; and (2) fixed class-per-client setting, where each client has access to only a fixed proportion of the total classes, i.e., $\#FC \in \{20\%, 40\%, 60\%\}$. The results demonstrate that ProtoGen and ProtoGen$^+$ consistently outperform the state-of-the-art methods across various datasets, with the performance gap widening as dataset complexity and heterogeneity increase. For example, while the performance boost of ProtoGen on the MNIST dataset is marginal, it becomes more pronounced on CIFAR-10 and even larger as heterogeneity (lower β value) rises. For instance, on CIFAR-10 dataset, when $\beta = 0.05$ we get the best test accuracy of ProtoGen as 76.8%, while the accuracy of the second best model FedProto is 75.18% . Similarly, when each client has only 20% of the total classes ($n = 0.2$), ProtoGen achieves 81.07% accuracy. Even in the most challenging CIFAR-100 scenarios (e.g., $\beta = 0.05$), ProtoGen delivers substantial gains, achieving up to 36.2% test accuracy, which is significantly higher than other baselines like FedProto (32.31%) and FedPVR (23.23%). Such noticeable improvement is seen in the results in almost all cases which demonstrates the effectiveness of ProtoGen. The superior performance of ProtoGen is largely attributed to its use of *PPGM*-based class feature representations, which, in a federated learning setting, can effectively approximate the feature distribution for each class. In contrast, methods like FedAvg, SCAFFOLD, and FedFA exhibit significant performance degradation as the data becomes more heterogeneous. While FedProto, leveraging class prototype-based representations, manages to mitigate some distribution challenges, ProtoGen outperforms it in nearly every test case due to the strength of its *PPGM*-based approach. Moreover, we find that prototype-driven methods consistently outperform others under extreme heterogeneity, particularly when $\beta \in \{0.1, 0.05\}$ and the fraction of participating clients per round is as low as 0.2%. This robustness can be attributed to the global prototype aggregation mechanism. Even under severe data imbalance, there typically exist a small subset of clients with relatively diverse local class distributions as observed in Figures 3, 4, and 5.

D. Visualization of Generated Features

Figure 6 presents the t-SNE visualization of features generated by clients' feature generation parameters at different rounds of training. The visualization is based on a label skew partition with $\beta = 0.5$ for the CIFAR-10 dataset. The result demonstrate that in the beginning of the round the generated features are scattered without clear class boundaries, indicating that the feature generation module has not yet learned meaningful class representations (Figure 6(a)). As training progresses to Round 30 (Figure 6(b)), the formation of distinct clusters is observed, which suggests that the feature generation module is starting to capture class-specific characteristics. By Round 100 (Figure 6(c)), the clusters become more pronounced and separated, demonstrating improved class

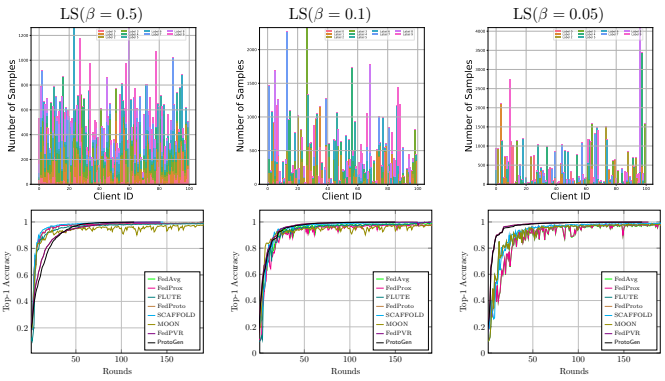


Fig. 3. Convergence comparison on MNIST under varying label skew (LS) settings.

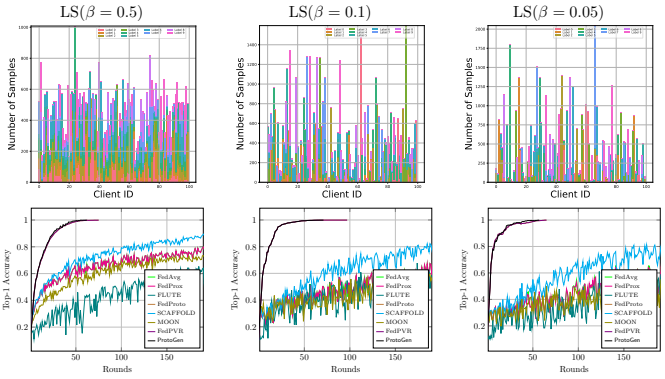


Fig. 4. Convergence comparison on CIFAR-10 under varying label skew (LS) settings.

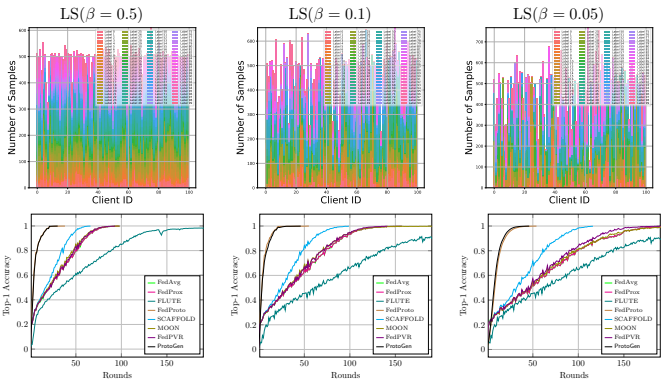


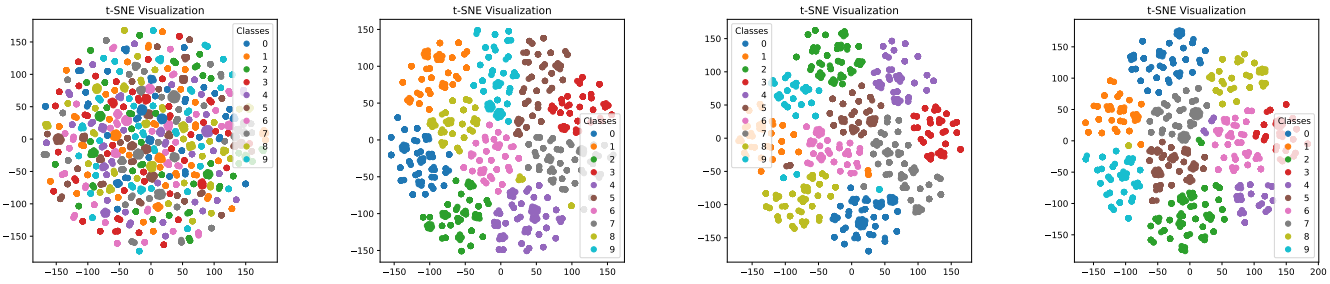
Fig. 5. Convergence comparison on CIFAR-100 under varying label skew (LS) settings.

discrimination. Finally, at Round 150 (Figure 6(d)), we observe a well-defined and tightly clustered class representations, indicating that ProtoGen has successfully learned to generate features that effectively capture the underlying class structure despite the heterogeneous data distribution.

E. Communication Efficiency

Communication efficiency is a critical concern in FL, especially under constrained network conditions and limited bandwidth. To evaluate this, in Tables I and II, we report the amount of data exchanged between clients and server per round, along with the number of global rounds required to converge. As observed, both ProtoGen and ProtoGen$^+$ consistently outperform the baselines in terms of convergence speed in all heterogeneous settings under consideration. While FedProto is converging faster among the baselines, MOON and FLUTE, on the other hand, have slower convergence suggesting the requirement of more communication rounds. Additionally, the results reveal that ProtoGen$^+$ and FedProto have the lowest communication cost per round due to their simple strategy of transmitting lightweight class-wise prototypes, whereas SCAFFOLD has the highest communication cost per round. On the other hand, ProtoGen demonstrates a significant communication efficiency, transmitting substantially less data per round compared to most baselines, including SCAFFOLD, FLUTE, MOON, and FedProx. This efficiency arises from ProtoGen's design where clients only share compact generative model parameters (decoder) with the server, instead of full model gradients or weights. It is worth to note that, even though ProtoGen$^+$ maintains equal communication cost per round as of FedProto, due to its faster convergence the overall communication cost becomes lower than FedProto.

a) Differential Privacy Analysis.: To evaluate the robustness of different methods under differential privacy (DP) constraints, we simulate increasing levels of Gaussian noise (σ_s^2) injected into client data representations. As shown in Figure 7, most baseline approaches, including FedAvg, FedProx, MOON, and FLUTE, experience a sharp decline in accuracy as the privacy noise increases. This drop is particularly severe for MOON and SCAFFOLD, which rely heavily on precise gradient or representation alignment, making them sensitive to noise perturbations. In contrast, ProtoGen exhibits a notably stable performance across all privacy levels, maintaining accuracy above 72% even at the highest noise level ($\sigma_s^2 = 0.5$). This resilience is largely attributed to its prototype-based generative mechanism, which aggregates noisy representations into a global class-wise structure, thereby smoothing out local noise through feature-level abstraction. Unlike gradient-sharing methods, ProtoGen avoids transmitting direct model updates, instead relying on semantically meaningful prototypes generated via lightweight decoders. These results suggest that ProtoGen not only provides strong privacy guarantees due to its minimal sharing and noise-injection strategy, but also preserves predictive accuracy better than other FL approaches under strict DP budgets. Therefore, it achieves a favorable trade-off between privacy preservation and model



(a) Round 0 (b) Round 30 (c) Round 100 (d) Round 150

Fig. 6. t-SNE visualization of the features generated by client's features generation parameters at different rounds of training. We consider label skew partition with $\beta = 0.5$ for CIFAR-10 dataset.

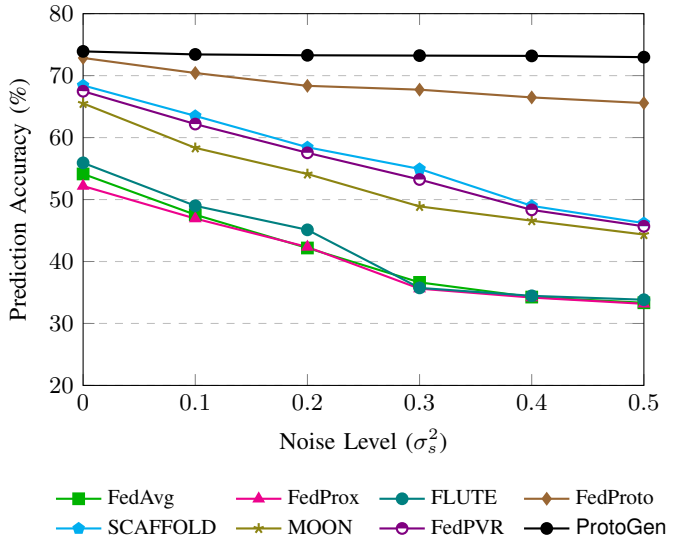


Fig. 7. Prediction accuracy (%) under varying levels of differential privacy noise (σ_s^2). The figure compares our approach with baseline methods under different strengths of DP guarantees.

utility, making it particularly suitable for privacy-sensitive FL applications.

F. Scalability of ProtoGen

To evaluate ProtoGen's scalability, we conduct experiments on the CIFAR-10 dataset using four distinct configurations that vary both the total client pool size and per-round participation rates. The results in Figure 8 demonstrate ProtoGen's robust performance across these diverse settings. With 200 total clients and only 10% participation per round ($\mathcal{K}$-200, $\mathcal{S}_t$-10), ProtoGen maintains strong performance despite the challenging sparse participation scenario. When further reducing participation to just 5% ($\mathcal{K}$-200, $\mathcal{S}_t$-10), ProtoGen continues to demonstrate resilience with a slightly reduced convergence trajectory. For the 100-client configurations, we observe accelerated initial convergence compared to the 200-client settings, attributable to the higher participation rates (20% and 10%, respectively). The $\mathcal{K}$-100, $\mathcal{S}_t$-20 setting exhibits particularly impressive performance, suggesting this balanced configuration effectively leverages ProtoGen's feature generation capabilities. Across all scenarios, ProtoGen shows consistent improvement as communication rounds progress. This sustained improvement trajectory indicates that ProtoGen's feature generation approach effectively accumulates knowledge over time, even with limited client participation. These results underscore ProtoGen's practical applicability in large-scale federated learning deployments.

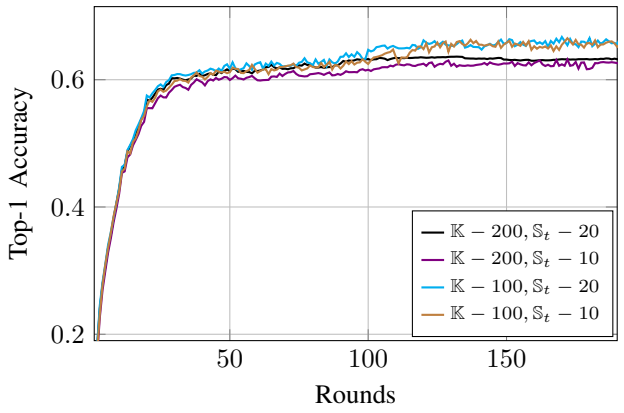


Fig. 8. Scalability Performance of ProtoGen with varying number of total clients ($\mathbb{K}$) and number of participants per round ($\mathbb{S}_t$) on CIFAR-10 under LS($\beta = 0.5$), using the ResNet-8 model.

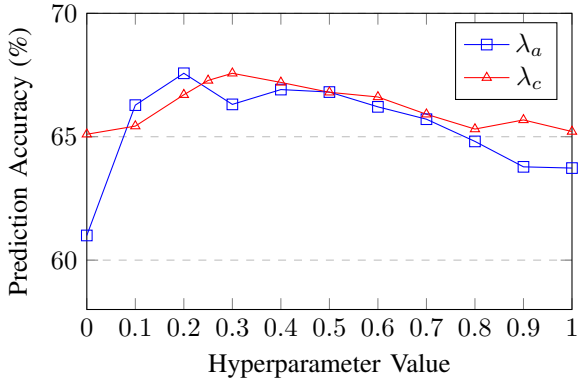


Fig. 9. Average test accuracy on CIFAR-10 using the ResNet-8 model under a Dirichlet data distribution among 100 clients ($\beta = 0.5$). Results are reported for varying λ_a with fixed $\lambda_c = 0.3$, and varying λ_c with fixed $\lambda_a = 0.2$.

G. Ablation Study

To understand the contribution of different components in our proposed ProtoGen framework, we conduct an ablation study by varying the hyper-parameters λ_a and λ_c, which control the strength of feature alignment and classifier regularization, respectively as mentioned in Equation 3. As shown in Figure 9, we observe that both hyper-parameters significantly impact the model's performance on the CIFAR-10 dataset. For λ_a, which controls the alignment between local features and global prototypes, we observe that the accuracy initially increases as λ_a increases from 0.01 to 0.5, reaching a peak of 66.89% at $\lambda_a = 0.2$. However, beyond this point, the accuracy decreases significantly, dropping to 65.31% at $\lambda_a = 0.5$. This suggests that while some degree of feature alignment is beneficial for addressing heterogeneity, excessive alignment can be detrimental, potentially forcing the model to discard client-specific information that is valuable for local predictions. Similarly, for λ_c, which regulates the classifier training on generated features, we observe a peak accuracy of 66.89% at $\lambda_c = 0.3$. The accuracy curve for λ_c is more gradual, with a slower decline after the peak compared to λ_a. This indicates that the classifier regularization component is more robust to hyperparameter variations, providing consistent benefits across a wider range of values. These results highlight the importance of carefully balancing the different components of ProtoGen to achieve optimal performance. The ablation study confirms that both feature alignment and classifier regularization contribute significantly to ProtoGen's effectiveness in heterogeneous federated learning environments, but their relative importance and optimal settings depend on the specific dataset and heterogeneity conditions.

VII. Conclusion

This paper presents a novel FL approach, aiming to mitigate the effect of data heterogeneity while improving communication efficiency and privacy preservation. By leveraging a feature generation module, ProtoGen enables clients to share only lightweight feature generation parameters with the server, rather than full local model parameters. This approach not only helps to mitigates privacy risks but also reduces communication overhead. To address the challenges of data heterogeneity, ProtoGen trains local models by aligning their hidden features with global feature prototypes, ensuring consistent intra-class representations while enhancing inter-class distinctions. Additionally, a privacy-preserving generative model is employed to allow clients to capture a more generalized local pattern from the dataset helping the server to construct a unbiased global representation. Experimental results demonstrates a significant performance improvement by an average of 4.76% over state-of-the-art FL methods and 2.61% with respect to the second best performer in terms of accuracy. This highlights ProtoGen's ability to enhance the performance of underrepresented classes in clients, leading to a more balanced and equitable learning outcome. Furthermore, ProtoGen exhibits faster convergence rates with reduced communication cost compared to existing methods, maintaining its performance advantages even in extreme heterogeneous settings. As an interesting note, we find ProtoGen$^+$ (a communication-efficient variant of ProtoGen) not only achieves performance comparable to ProtoGen, but also reduces communication overhead by an average of 84.44% compared to ProtoGen. However, this comes at the cost of some compromise in privacy.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," *arXiv preprint arXiv:1602.05629*, 2017.

[2] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-iid data," *arXiv preprint arXiv:1806.00582*, 2018.

[3] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *International Conference on Learning Representations*, 2021.

[4] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," *International Conference on Machine Learning*, pp. 5132–5143, 2020.

[5] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[6] S. J. Reddi, Z. Charles, *et al.*, "Adaptive federated optimization," in *International Conference on Learning Representations*, 2021.

[7] J. Wang *et al.*, "Tackling the objective inconsistency problem in heterogeneous federated optimization," *Advances in neural information processing systems*, vol. 33, pp. 7611–7623, 2020.

[8] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečnỳ, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," *arXiv preprint arXiv:2003.00295*, 2021.

[9] D. Chen and otthers, "Elastic aggregation for federated optimization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12187–12197, 2023.

[10] D. Chen, J. Hu, V. J. Tan, X. Wei, and E. Wu, "Elastic aggregation for federated optimization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12187–12197, June 2023.

[11] Z. Yang, Y. Zhang, Y. Zheng, X. Tian, H. Peng, T. Liu, and B. Han, "Fedfed: Feature distillation against data heterogeneity in federated learning," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[12] Y. Zhang, J. C. Duchi, and M. J. Wainwright, "Federated learning with non-iid data," *arXiv preprint arXiv:2007.13518*, 2020.

[13] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, "Federated learning with heterogeneous architectures using graph hypernetworks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 1243–1256, 2022.

[14] R. Liu, C. Shen, and J. Yang, "Federated representation learning in the under-parameterized regime," in *Forty-first International Conference on Machine Learning*, 2024.

[15] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," *IEEE Symposium on Security and Privacy (SP)*, 2019.

[16] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," *IEEE Symposium on Security and Privacy (SP)*, 2019.

[17] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, "Beyond inferring class representatives: User-level privacy leakage from federated learning," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, pp. 2512–2520, IEEE, 2019.

[18] Y. Tan, W. Xu, Y. Wang, and J. Feng, "Fedproto: Federated prototype learning across heterogeneous clients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 8482–8490, 2022.

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[20] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.

[21] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10713–10722, 2021.

[22] T. Zhou, J. Zhang, and D. K. Tsang, "Fedfa: Federated learning with feature anchors to align features and classifiers for heterogeneous data," *IEEE Transactions on Mobile Computing*, vol. 23, pp. 6731–6742, jun 2024.

[23] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "Fedbn: Federated learning on non-iid features via local batch normalization," *arXiv preprint arXiv:2102.07623*, 2021.

[24] B. Li, M. N. Schmidt, T. S. Alstrøm, and S. U. Stich, "On the effectiveness of partial variance reduction in federated learning with heterogeneous data," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3964–3973, June 2023.

[25] C. Mclaughlin and L. Su, "Personalized federated learning via feature distribution adaptation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 77038–77059, 2024.

[26] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Commun. ACM*, vol. 63, p. 139–144, Oct. 2020.

[27] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020.

[28] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science, now Publishers Inc., 2014.

[29] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[30] N. Bouguila, D. Ziou, and J. Vaillancourt, "Novel mixtures based on the dirichlet distribution: Application to data and image classification," vol. 2734, pp. 172–181, 07 2003.